

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: Mukesh Raj L

REG NO: 192571036

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)

Assessment Tool Weightage: 35%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze	5
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze	5
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply	5
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze	5
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply	5
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze	5
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply	5

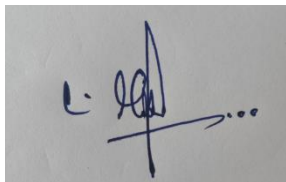
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply	5
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze	5
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze	5

Code of Conduct:

I Mukesh Raj L(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1)

Query Execution Plan and Heuristic Optimization

1. Example SQL Query

Consider three relations:

- EMPLOYEE(EmpID, Name, DeptID, Salary)
- DEPARTMENT(DeptID, DeptName)
- PROJECT(EmpID, ProjectName)

```
SELECT E.Name, D.DeptName
FROM EMPLOYEE E
JOIN DEPARTMENT D ON E.DeptID = D.DeptID
WHERE E.Salary > 50000
AND E.EmpID IN
(
    SELECT P.EmpID
    FROM PROJECT P
    WHERE P.ProjectName = 'AI Project'
);
```

This query contains:

- A **join** between EMPLOYEE and DEPARTMENT
- A **subquery** on PROJECT
- A **selection condition** on salary
- A **selection condition** on project name
- A **projection** of only required attributes

2. Initial Query Processing

The database initially identifies the operations required.

```
PROJECT
↓
Select ProjectName = 'AI Project'
↓
Find matching EmpID
↓
EMPLOYEE
↓
Select Salary > 50000
↓
```

Join with matching project employees

↓

Join with DEPARTMENT

↓

Project Name and DeptName

The query can be represented in relational algebra as:

```

$$\pi \text{ Name, DeptName}$$

$$(\sigma \text{ Salary} > 50000$$

$$(\text{EMPLOYEE} \bowtie \text{DEPARTMENT}$$

$$)$$

$$\bowtie$$

$$\sigma \text{ ProjectName} = \text{'AI Project'}$$

$$(\text{PROJECT}$$

$$)$$

$$)$$

```

The symbol π represents **projection**, σ represents **selection**, and $\bowtie$ represents **join**.

In this plan, large intermediate results may be produced before unnecessary records are removed.

This can increase:

- CPU processing
- Memory usage
- Disk I/O
- Join processing time

Therefore, **heuristic optimization** is applied.

4. Heuristic Optimization Rules

Heuristic optimization uses predefined rules to improve the query execution plan.

The important rules are:

Rule 1: Push Selections Down

Selection operations should be performed as early as possible.

Instead of joining all employee records first:

EMPLOYEE ⋈ DEPARTMENT



Salary > 50000

Apply the selection first:

EMPLOYEE



Salary > 50000



Selected EMPLOYEE records



JOIN

Similarly:

PROJECT



ProjectName = 'AI Project'



Selected PROJECT records

This reduces the number of tuples before performing joins.

Rule 2: Push Projections Down

Only the required attributes should be carried through later operations.

From EMPLOYEE, only required attributes are:

EmpID, Name, DeptID

From DEPARTMENT:

DeptID, DeptName

From PROJECT:

EmpID

Instead of processing all attributes:

EMPLOYEE(EmpID, Name, DeptID, Salary, ...)

the optimizer can reduce unnecessary columns after applying the needed selection.

This reduces intermediate result size.

Rule 3: Perform Selective Operations First

The operation that removes the maximum number of unnecessary tuples should be performed early.

For example:

PROJECT

↓

ProjectName = 'AI Project'

may reduce thousands of project records to only a small number of matching employees.

Similarly:

EMPLOYEE

↓

Salary > 50000

removes employees who do not satisfy the condition.

Therefore, joins are performed only on relevant records.

Rule 4: Replace Subquery with an Efficient Join

The subquery:

E.EmpID IN

(

SELECT P.EmpID

FROM PROJECT P

WHERE P.ProjectName = 'AI Project'

)

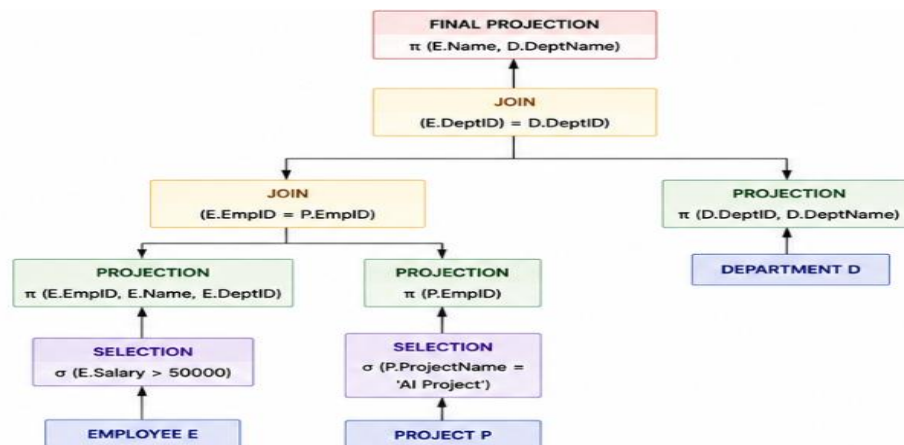
can conceptually be processed by first selecting the required PROJECT records and then joining using EmpID.

PROJECT
 ↓
 σ ProjectName = 'AI Project'
 ↓
 π EmpID
 ↓
 JOIN with EMPLOYEE

This avoids repeatedly considering unnecessary project records.

5. Optimized Query Execution Plan

After applying heuristic optimization, the execution plan becomes:



6. Step-by-Step Optimized Execution

Step 1: Select required project records

PROJECT
 ↓
 σ (ProjectName = 'AI Project')
 ↓
 Matching Project Records

Only employees working on the required project are selected.

Step 2: Project only EmpID

Matching Project Records



$\pi(\text{EmpID})$



Required Employee IDs

Unnecessary project attributes are removed.

Step 3: Select employees with required salary

EMPLOYEE



$\sigma(\text{Salary} > 50000)$



High Salary Employees

Employees not satisfying the salary condition are removed before joining.

Step 4: Project required employee attributes

High Salary Employees



$\pi(\text{EmpID}, \text{Name}, \text{DeptID})$

Only attributes needed for future joins and final output are retained.

Step 5: Join Employee and Project Results

Selected Employees

$\bowtie \text{EmpID}$

Selected Project Employees

This identifies employees satisfying both conditions.

Step 6: Join with Department

Employee Result

$\bowtie \text{DeptID}$

Department

The department name is obtained only for the reduced set of employee records.

Step 7: Final Projection

Finally:

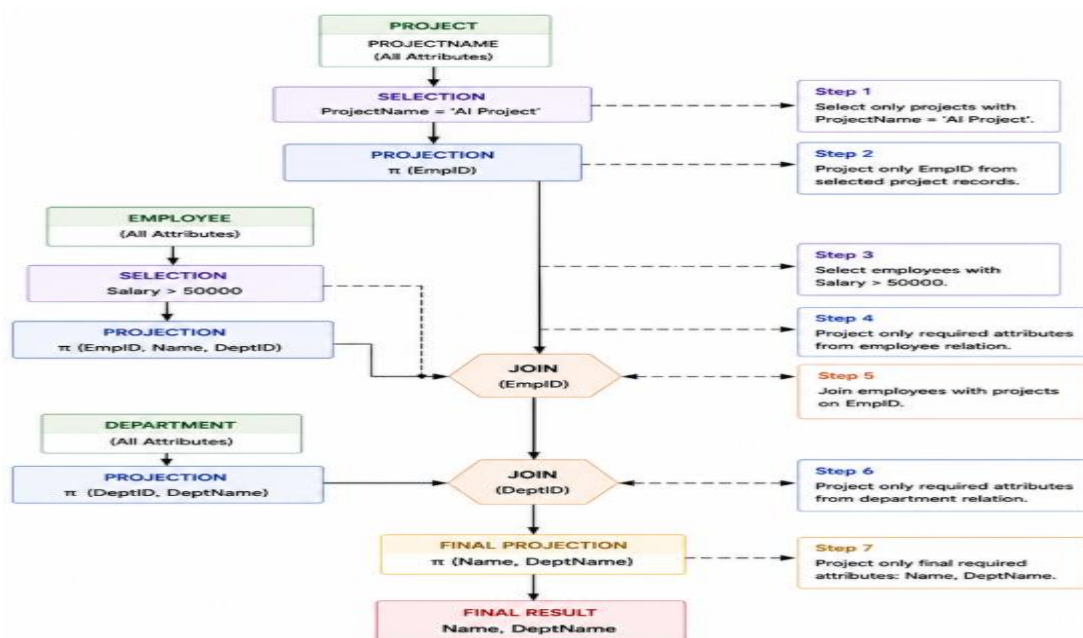
$\pi(\text{Name}, \text{DeptName})$

Only the attributes required by the SQL query are returned.

7. Comparison of Plans

Aspect	Initial Plan	Optimized Plan
Selection	Performed later	Performed early
Projection	Mostly at the end	Pushed downward
Intermediate tuples	Large	Reduced
Join cost	Higher	Lower
Memory requirement	Higher	Lower
Disk I/O	More	Reduced
Query efficiency	Lower	Higher

8. Final Optimized Flow



2)

Cost-Based Optimization for Join Operations

1. Introduction to Cost-Based Optimization

Cost-based query optimization selects the query execution method with the **lowest estimated cost**.

For join operations, the database optimizer considers factors such as:

- Number of tuples
- Number of pages or blocks
- Available memory buffers
- Whether relations are already sorted
- Presence of indexes
- Disk I/O cost

Suppose two relations are:

R = Outer Relation

S = Inner Relation

$B(R)$ = Number of pages of R

$B(S)$ = Number of pages of S

$T(R)$ = Number of tuples of R

$T(S)$ = Number of tuples of S

The optimizer estimates the cost of each join algorithm and chooses the one with the minimum cost.

2. Nested-Loop Join

In a **Nested-Loop Join**, one relation is chosen as the outer relation, and for every tuple or page in that relation, the other relation is scanned.

Basic Working

Relation R



Take one record/page



Scan Relation S



Compare join condition



Repeat for all records/pages of R

Cost

For a simple page nested-loop approach, the approximate cost is:

$$\text{Cost} \approx B(R) + B(R) \times B(S)$$

If the relations are swapped:

$$\text{Cost} \approx \text{B}(\text{S}) + \text{B}(\text{S}) \times \text{B}(\text{R})$$

Therefore, it is usually better to choose the **smaller relation as the outer relation**.

Advantages

- Simple to implement.
- Suitable for small relations.
- Can be useful when an index exists on the join attribute.

Limitations

- Can require many repeated scans.
- High disk I/O for large relations.
- Usually inefficient for two large unsorted relations.

3. Sort-Merge Join

In a **Sort-Merge Join**, both relations are first sorted based on the join attribute. After sorting, the sorted relations are scanned and matching values are merged.

Working

$$\begin{array}{l} \text{Relation R} \rightarrow \text{Sort on Join Key } \Uparrow \\ \quad \quad \quad \vdash \rightarrow \text{Merge} \rightarrow \text{Result} \\ \text{Relation S} \rightarrow \text{Sort on Join Key } \Downarrow \end{array}$$

Step-by-Step Process

- Step 1 → Sort R
- Step 2 → Sort S
- Step 3 → Compare sorted join keys
- Step 4 → Merge matching records
- Step 5 → Produce result

Cost

The cost includes:

$$\begin{aligned} & \text{Sorting Cost of R} \\ & + \\ & \text{Sorting Cost of S} \end{aligned}$$

+

Merge/Scan Cost

In symbolic form:

$$\text{Cost} \approx \text{Sort}(R) + \text{Sort}(S) + B(R) + B(S)$$

The exact sorting cost depends on available memory and the external sorting algorithm used.

Advantages

- Very useful for large relations.
- Efficient when both relations are already sorted.
- Suitable for ordered processing.
- Can handle range-related operations effectively.

Limitations

- Sorting can be expensive.
- Less attractive when the relations are unsorted and sorting requires significant disk I/O.

4. Hash Join

A **Hash Join** is mainly suitable for **equality join conditions**, such as:

$$R.ID = S.ID$$

It normally works in two phases.

Phase 1: Build/Partition Phase

Records are distributed into hash buckets.

$R \rightarrow \text{Hash Function} \rightarrow \text{Buckets}$

$S \rightarrow \text{Same Hash Function} \rightarrow \text{Buckets}$

Phase 2: Matching Phase

Corresponding buckets are compared.

$R \text{ Bucket } 1 \leftrightarrow S \text{ Bucket } 1$

$R \text{ Bucket } 2 \leftrightarrow S \text{ Bucket } 2$

$R \text{ Bucket } 3 \leftrightarrow S \text{ Bucket } 3$

Approximate Cost

For a basic hash join, the approximate disk I/O cost is often expressed as:

$$\text{Cost} \approx 3 \times (B(R) + B(S))$$

The exact cost can vary depending on memory availability and the hash-join implementation.

Advantages

- Efficient for large relations.
- Very effective for equality joins.
- Does not require sorting.
- Often has lower cost than nested-loop join for large unsorted relations.

Limitations

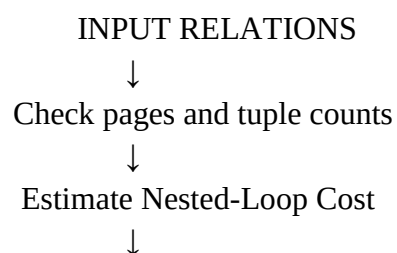
- Mainly suitable for equality conditions.
- Performance depends on a good hash distribution.
- Less suitable for non-equality and range joins.

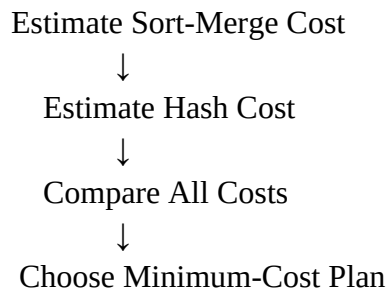
5. Cost Comparison

Join Method	Main Operation	Approximate Cost	Best Suitable Situation
Nested-Loop Join	Repeated scanning	$B(R) + B(R) \times B(S)$	Small relation or indexed join
Sort-Merge Join	Sort + merge	$\text{Sort}(R) + \text{Sort}(S) + B(R) + B(S)$	Sorted data / large ordered joins
Hash Join	Hash partition + match	$\approx 3(B(R)+B(S))$	Large equality joins

6. How Cost-Based Optimization Chooses the Best Plan

The database optimizer follows these steps:





In mathematical form:

Optimal Join Plan =
MIN
(
Cost of Nested-Loop Join,
Cost of Sort-Merge Join,
Cost of Hash Join
)

7. Example of Optimization Decision

The optimizer performs a comparison like this:

Nested-Loop Cost → C1
Sort-Merge Cost → C2
Hash Join Cost → C3

Then:

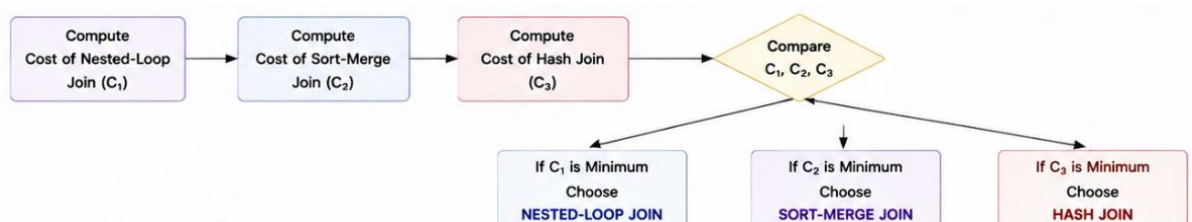
If C1 is minimum → Choose Nested-Loop Join

If C2 is minimum → Choose Sort-Merge Join

If C3 is minimum → Choose Hash Join

Thus, the decision is based on the **estimated execution cost**, rather than simply using one fixed join method.

8. Final Decision Rules



The final choice should always depend on the actual **tuple counts and page sizes** specified for the relations. The assessment question requires such values, but those values are not present in the uploaded file, so a specific numerical winner cannot be calculated from the source.

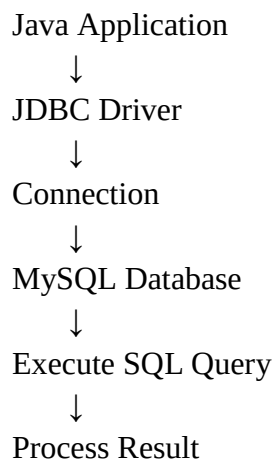
3)

JDBC Connectivity and PreparedStatement

1. Introduction

JDBC (Java Database Connectivity) is an API that allows a Java program to connect and interact with a database such as MySQL.

The basic JDBC process is:



2. Requirements

For JDBC connectivity, we need:

- Java program
- MySQL database
- MySQL JDBC Driver
- Database URL
- Username and password

Example database details:

Database Name : college_db

Table Name : STUDENT

Username : root

Password : root

Assume the table is:

STUDENT

```
-----  
SID | Name | Department | CGPA  
-----  
101 | Arun | CSE      | 8.5  
102 | Bala | ECE      | 8.0  
103 | Ravi | CSE      | 9.0
```

3. JDBC Steps

Step 1: Import JDBC Package

```
import java.sql.*;
```

This provides classes such as:

- Connection
- DriverManager
- PreparedStatement
- ResultSet

Step 2: Establish Database Connection

The Java program connects to MySQL using:

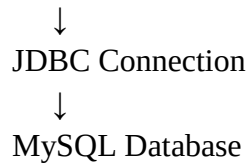
```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/college_db",  
    "root",  
    "root"  
);
```

Structure:

Java Program



DriverManager



4. PreparedStatement

A **PreparedStatement** is used to execute a parameterized SQL query.

Instead of writing:

```
SELECT * FROM STUDENT WHERE SID = 101;
```

we use a placeholder:

```
SELECT * FROM STUDENT WHERE SID = ?;
```

The value is supplied using Java:

```
ps.setInt(1, 101);
```

Here:

? → Parameter placeholder
1 → First parameter
101 → Value assigned

This makes the query safer and reusable.

5. Java Program for JDBC Connectivity

```
import java.sql.*;
```

```
public class JDBCPreparedStatement {  
    public static void main(String[] args) {
```

```
        String url = "jdbc:mysql://localhost:3306/college_db";  
        String username = "root";  
        String password = "root";
```

```
        String query = "SELECT * FROM STUDENT WHERE SID = ?";
```

```
        try {  
            // Step 1: Establish connection  
            Connection con = DriverManager.getConnection(  

```

```

        url, username, password);

System.out.println("Database Connected Successfully");

// Step 2: Create PreparedStatement
PreparedStatement ps = con.prepareStatement(query);

// Step 3: Set parameter value
ps.setInt(1, 101);

// Step 4: Execute query
ResultSet rs = ps.executeQuery();

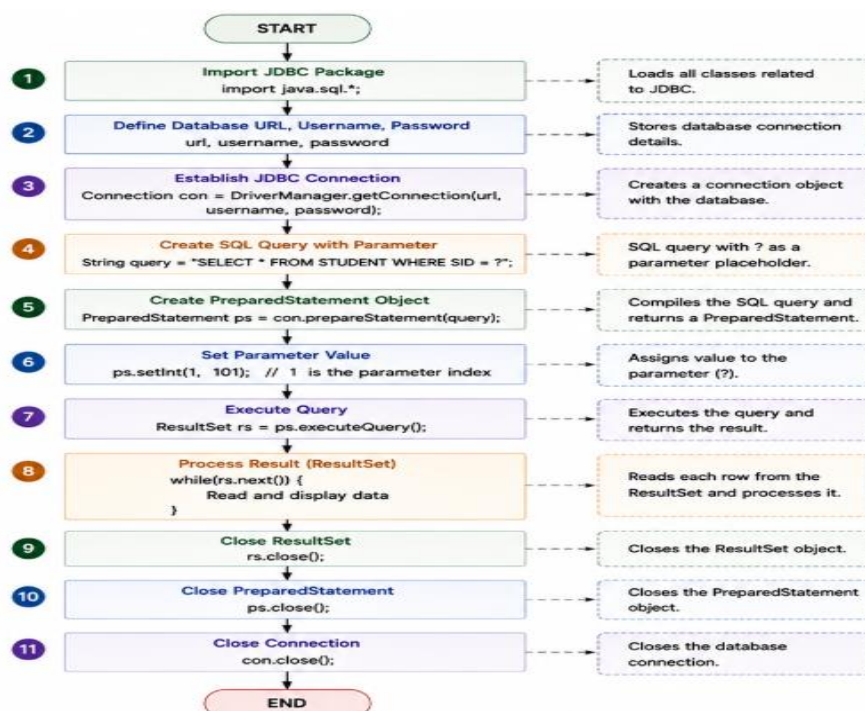
// Step 5: Display result
while (rs.next()) {
    System.out.println("SID: " + rs.getInt("SID"));
    System.out.println("Name: " + rs.getString("Name"));
    System.out.println("Department: "
        + rs.getString("Department"));
    System.out.println("CGPA: " + rs.getDouble("CGPA"));
}

// Step 6: Close resources
rs.close();
ps.close();
con.close();

} catch (Exception e) {
    System.out.println(e);
}
}
}

```

6. Execution Flow



7. Advantages of PreparedStatement

Feature	Benefit
Parameterized query	Values can be changed easily
Security	Helps prevent SQL injection
Reusability	Same query can be executed with different values
Performance	Query can be reused efficiently
Readability	SQL code is easier to manage
Type handling	Methods such as setInt() and setString() handle values appropriately

4)

Conflict Serializability Using Precedence Graph

1. Conflict Serializability

A schedule is **conflict-serializable** if it can be transformed into an equivalent serial schedule by swapping non-conflicting operations.

Two operations conflict when:

1. They belong to **different transactions**.
2. They access the **same data item**.

3. At least one operation is a **write**.

The conflicting operation pairs are:

Read-Write (R-W)

Write-Read (W-R)

Write-Write (W-W)

Two read operations do not conflict:

Read-Read (R-R) → No Conflict

2. Illustrative Schedule

Consider four transactions T1, T2, T3, and T4.

Time	T1	T2	T3	T4
1	R(A)			
2		W(A)		
3			R(B)	
4			W(B)	
5	W(C)			
6		R(C)		
7			W(D)	
8			R(D)	

The transactions operate concurrently on different data items.

3. Identify Conflicting Operations

We compare operations performed on the same data item.

Conflict 1: Data Item A

T1: R(A)

T2: W(A)

Since R(A) of T1 occurs before W(A) of T2:

T1 → T2

Conflict 2: Data Item B

T3: R(B)
T4: W(B)

Since T3 performs the operation first:

$T3 \rightarrow T4$

Conflict 3: Data Item C

T1: W(C)
T2: R(C)

Since the write operation of T1 occurs before the read operation of T2:

$T1 \rightarrow T2$

This edge already exists.

Conflict 4: Data Item D

T3: W(D)
T4: R(D)

Therefore:

$T3 \rightarrow T4$

This edge also already exists.

4. Construct the Precedence Graph

A **precedence graph** contains:

- One node for each transaction.
- A directed edge $T_i \rightarrow T_j$ when an operation of T_i conflicts with and occurs before an operation of T_j .

5. Check for a Cycle

The important rule is:

Precedence Graph has NO cycle



Conflict Serializable

Precedence Graph has a cycle



NOT Conflict Serializable

In this example:

$T1 \rightarrow T2$

$T3 \rightarrow T4$

There is **no cycle**.

Therefore:

Schedule = Conflict Serializable

6. Find the Equivalent Serial Order

Since the graph has no cycle, a valid serial order can be obtained through **topological ordering**.

The following conditions must be maintained:

T1 before T2

T3 before T4

One possible serial order is:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4$

Another valid order is:

$T3 \rightarrow T4 \rightarrow T1 \rightarrow T2$

Any serial order that preserves the precedence relationships is valid.

7. Example of a Non-Serializable Schedule

Consider:

T1: W(A)

T2: R(A)

T2: W(B)

T1: R(B)

The conflicts produce:

T1 $\rightarrow$ T2 from A

T2 $\rightarrow$ T1 from B

5)

Basic Two-Phase Locking (2PL) and Deadlock Handling

1. Basic Two-Phase Locking

Two-Phase Locking (2PL) is a concurrency-control protocol used to ensure **conflict serializability**.

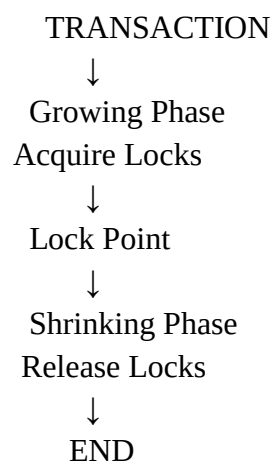
It has two phases:

Phase 1 – Growing Phase

- A transaction can **acquire locks**.
- It cannot release any lock.

Phase 2 – Shrinking Phase

- A transaction can **release locks**.
- It cannot acquire any new lock.



2. Lock Types

Two common locks are:

Shared Lock (S)

Used for reading.

S-lock $\rightarrow$ Read

Multiple transactions can hold shared locks on the same item simultaneously.

Exclusive Lock (X)

Used for writing.

X-lock $\rightarrow$ Write

3.Example Transaction Set

Consider:

T1: Read(A)
Write(B)

T2: Read(B)
Write(A)

Assume both transactions need exclusive locks for writing.

Execution

T1: Lock-X(A)
Read(A)
Write(A)

T2: Lock-X(B)
Read(B)
Write(B)

Now:

T1 holds X(A)
T2 holds X(B)

T1 requests X(B) $\rightarrow$ WAIT
T2 requests X(A) $\rightarrow$ WAIT

This produces a deadlock.

4. How to Detect Deadlock

The database can construct a **Wait-For Graph**.

Transaction $\rightarrow$ Transaction

If there is a cycle:

$T1 \rightarrow T2 \rightarrow T1$

then:

Cycle detected

↓

Deadlock exists

If there is no cycle:

$T1 \rightarrow T2 \rightarrow T3$

then there is no deadlock.

5. Resolving the Deadlock

One common solution is to **abort/rollback one transaction**.

Suppose T2 is selected as the victim.

T1 holds X(A)

T2 holds X(B)

T1 $\rightarrow$ waits for B

T2 $\rightarrow$ waits for A

Abort T2:

T2 $\rightarrow$ ROLLBACK

↓

Release X(B)

↓

T1 obtains X(B)

↓

T1 continues

↓

COMMIT

After T2 releases its lock:

T1
↓
Acquire B
↓
Complete operations
↓
Commit

Then T2 can be restarted later.

6. Correct 2PL Execution

A possible execution is:

T1:
LOCK-X(A)
LOCK-X(B)
READ(A)
WRITE(A)
WRITE(B)
UNLOCK(A)
UNLOCK(B)
COMMIT

Here, all required locks are acquired during the **growing phase** before any lock is released.

Growing Phase

LOCK-X(A)
LOCK-X(B)
READ/WRITE

↓

Shrinking Phase

UNLOCK(A)
UNLOCK(B)

↓

COMMIT

Once UNLOCK(A) occurs, T1 cannot acquire another new lock.

7. Important Rules of Basic 2PL

1. A transaction must acquire locks before accessing the corresponding data item.
2. During the growing phase, locks can only be acquired.
3. Once a transaction releases its first lock, it enters the shrinking phase.
4. No new lock can be acquired during the shrinking phase.
5. Basic 2PL guarantees **conflict serializability**.
6. Basic 2PL does **not completely prevent deadlocks**.

8. Deadlock Resolution Methods

Deadlocks can be handled using:

1. Deadlock Detection and Recovery

Build Wait-For Graph



Check for Cycle



Cycle?



Yes



Abort Victim
Transaction



No



Continue

2. Transaction Rollback

One transaction is rolled back to release its locks.

3. Lock Ordering

Transactions request locks in a predefined order.

For example:

Always lock A before B

Then:

T1: Lock A → Lock B

T2: Lock A $\rightarrow$ Lock B

This can prevent circular waiting.

9. Advantages and Limitation

Feature	Basic 2PL
Concurrency control	Yes
Conflict serializability	Guaranteed
Locking mechanism	S/X locks
Deadlock prevention	No
Deadlock detection	Can be used
Deadlock recovery	Rollback/abort
Main advantage	Ensures serializable schedules
Main limitation	Deadlocks may occur

6)

Wait-Die and Wound-Wait Deadlock Prevention

1. Introduction

Both **Wait-Die** and **Wound-Wait** are **timestamp-based deadlock prevention schemes**.

Each transaction receives a timestamp when it starts:

Older Transaction $\rightarrow$ Smaller Timestamp

Younger Transaction $\rightarrow$ Larger Timestamp

For example:

T1 = TS 10

T2 = TS 20

T3 = TS 30

Therefore:

T1 $\rightarrow$ Oldest

T2 $\rightarrow$ Younger than T1

T3 $\rightarrow$ Youngest

When a transaction requests a lock already held by another transaction, the timestamp is used to decide whether the transaction should:

- Wait
- Abort / Rollback

2. Wait-Die Scheme

The rule is:

Older Transaction requests lock held by Younger

↓
WAIT

Younger Transaction requests lock held by Older

↓
DIE
(Abort/Rollback)

Rule Table

Requesting Transaction	Lock-Holding Transaction	Action
Older	Younger	Wait
Younger	Older	Abort (Die)

The important idea is:

Older → Wait

Younger → Die

3. Wait-Die Example

Assume:

T1 = TS 10 → Older

T2 = TS 20 → Younger

Case 1: T1 Requests a Lock Held by T2

T1 requests X(A)

↓
A is locked by T2
↓
T1 is older than T2
↓
T1 WAITS

Result:

T1 → WAIT
T2 → Continues

Case 2: T2 Requests a Lock Held by T1

T2 requests X(A)
↓
A is locked by T1
↓
T2 is younger than T1
↓
T2 DIES
↓
Rollback

Result:

T2 → ABORT

After rollback, T2 may be restarted later.

4. Wound-Wait Scheme

The rule is the opposite:

Older Transaction requests lock held by Younger
↓
WOUND
Younger transaction aborts

Younger Transaction requests lock held by Older
↓
WAIT

Rule Table

Requesting Transaction	Lock-Holding Transaction	Action
Older	Younger	Abort younger transaction
Younger	Older	Wait

The important idea is:

Older $\rightarrow$ Wounds

Younger $\rightarrow$ Waits

5. Wound-Wait Example

Again, assume:

T1 = TS 10 $\rightarrow$ Older

T2 = TS 20 $\rightarrow$ Younger

Case 1: T1 Requests a Lock Held by T2

T1 requests X(A)

↓

A is locked by T2

↓

T1 is older than T2

↓

T1 WOUNDS T2

↓

T2 ABORTS

↓

Lock is released

↓

T1 gets the lock

Result:

T1 $\rightarrow$ Continues

T2 $\rightarrow$ Abort / Rollback

Case 2: T2 Requests a Lock Held by T1

T2 requests X(A)

↓

A is locked by T1

↓

T2 is younger than T1

↓
T2 WAITS

Result:

T2 → WAIT

6. Comparison of Wait-Die and Wound-Wait

Situation	Wait-Die	Wound-Wait
Older requests lock from younger	Older waits	Younger aborts
Younger requests lock from older	Younger aborts	Younger waits
Transaction action	Wait or Die	Wound or Wait
Deadlock prevention	Yes	Yes
Based on	Timestamp	Timestamp

7. Why These Schemes Prevent Deadlocks

Deadlock requires transactions to wait in a circular manner.

For example:

T1 waits for T2

T2 waits for T3

T3 waits for T1

Both schemes use timestamps to control waiting and aborting. Therefore, transactions do not wait arbitrarily, which prevents circular waiting and hence prevents deadlock.

Timestamp Rules

↓
Controlled Waiting
↓
No Circular Waiting
↓
Deadlock Prevention

7)

Immediate Update Recovery Protocol

In the **Immediate Update** technique, database changes may be written to the database **before a transaction commits**.

Therefore, after a system crash, the recovery process may require:

- **REDO** – Repeat changes of committed transactions.
- **UNDO** – Reverse changes of uncommitted transactions.

Thus:

Immediate Update



System Failure



Identify Transactions



REDO committed transactions



UNDO uncommitted transactions

2. Transaction Log

A log record generally stores:

<Transaction, Data Item, Old Value, New Value>

For example:

<T1, A, 1000, 900>

This means:

Transaction : T1

Data Item : A

Old Value : 1000

New Value : 900

The old value is useful for **UNDO**, while the new value is useful for **REDO**.

3. Example Transaction Log

Consider the following log:

<START T1>

<T1, A, 1000, 900>

<START T2>
<T2, B, 500, 700>

<COMMIT T1>

<CHECKPOINT>

<START T3>
<T3, C, 300, 400>

<SYSTEM CRASH>

Now we must recover the database.

4. Step 1: Identify the Last Checkpoint

The log contains:

<CHECKPOINT>

The checkpoint helps the recovery system identify the point from which recovery processing is required.

Transaction Log



Find Last Checkpoint



Analyze Transactions

In the example, we examine the transactions around the checkpoint and crash.

5. Step 2: Identify Committed Transactions

From the log:

T1 → COMMIT

Therefore:

T1 = Committed Transaction

The changes of committed transactions must be preserved.

Hence:

T1 → REDO if required

6. Step 3: Identify Uncommitted Transactions

The log shows:

T2 → No COMMIT

T3 → No COMMIT

Therefore:

T2 = Uncommitted

T3 = Uncommitted

Their changes should not remain in the recovered database.

Hence:

T2 → UNDO

T3 → UNDO

7. REDO Operation

For committed transaction T1:

<T1, A, 1000, 900>

REDO applies the new value:

A = 900

Therefore:

T1 committed

↓

REDO

↓

A ← 900

8. UNDO Operation

For T2:

<T2, B, 500, 700>

Since T2 did not commit, restore the old value:

B = 500

For T3:

<T3, C, 300, 400>

Since T3 did not commit:

C = 300

Thus:

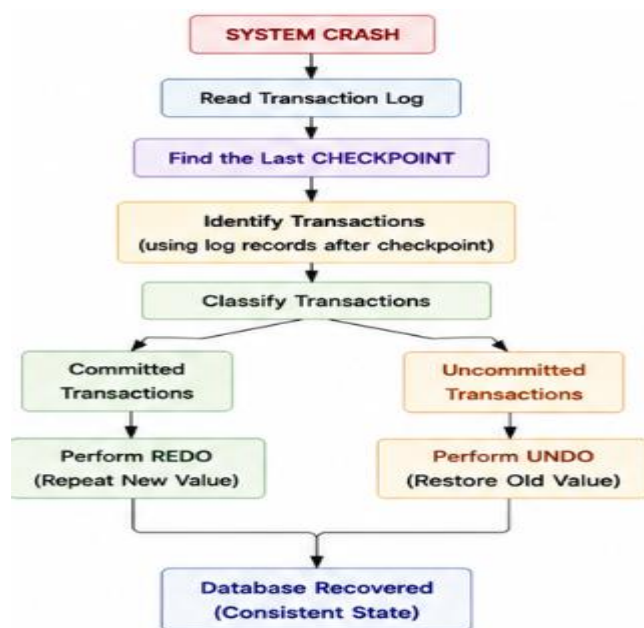
T2 → UNDO → B ← 500

T3 → UNDO → C ← 300

9. Recovery Table

Transaction	Status at Crash	Recovery Action
T1	Committed	REDO
T2	Uncommitted	UNDO
T3	Uncommitted	UNDO

10. Recovery Process



8)

Deferred Update (NO-UNDO/REDO) Recovery Technique

1. Introduction

In the **Deferred Update** recovery technique, database updates are **not written to the database until the transaction commits**.

Therefore, after a system failure:

- **Committed transactions** → **REDO**
- **Uncommitted transactions** → **No action**
- **UNDO is not required**

Hence, it is called:

NO-UNDO / REDO

The basic idea is:

Transaction



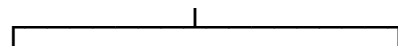
Perform Updates



Store Changes in Log



Commit?



Yes

No



Updates Applied No Database Update



REDO if needed No Action after Crash

2. Transaction Log

Consider the following illustrative log:

<START T1>

<T1, A, 900>

<START T2>

<T2, B, 700>

<COMMIT T1>

<START T3>

<T3, C, 400>

<SYSTEM CRASH>

Here:

- T1 has committed.
- T2 has not committed.
- T3 has not committed.

3. Step 1: Identify Transaction Status

From the log:

T1 → COMMITTED

T2 → UNCOMMITTED

T3 → UNCOMMITTED

This gives:

Transaction	Status
T1	Committed
T2	Uncommitted
T3	Uncommitted

4. Step 2: Apply Deferred Update Rule

The recovery rule is:

Committed Transaction



REDO

Uncommitted Transaction



NO ACTION

Therefore:

T1 → REDO
T2 → No Action
T3 → No Action

Unlike Immediate Update recovery, **UNDO is not performed** because uncommitted changes were not written to the database.

5. REDO Operation

The log record for T1 is:

<T1, A, 900>

Since T1 committed:

T1 → REDO
↓
A ← 900

Thus, the update of the committed transaction is ensured in the database.

6. Handling Uncommitted Transactions

For T2:

<T2, B, 700>

Since there is no:

<COMMIT T2>

T2 is uncommitted.

Therefore:

T2 → No REDO
T2 → No UNDO

Similarly, for T3:

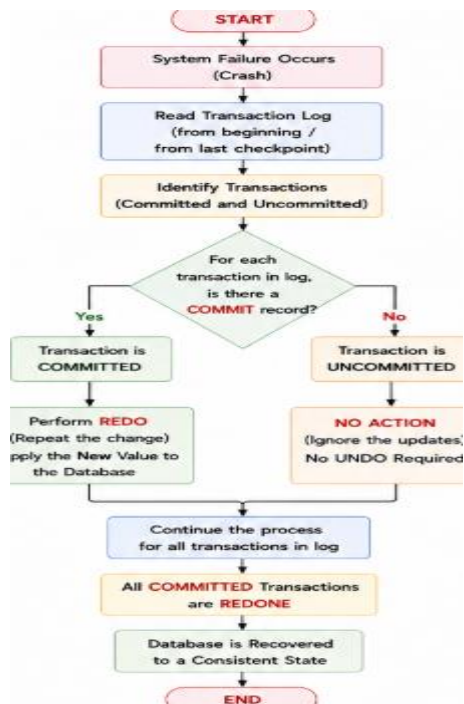
<T3, C, 400>

Since there is no commit:

T3 → No REDO
T3 → No UNDO

Their updates are simply ignored during recovery.

7. Recovery Process



8. Recovery Table

Transaction	Commit Status	Recovery Action
T1	Committed	REDO
T2	Uncommitted	No Action
T3	Uncommitted	No Action

Therefore:

T1 → REDO

T2 → IGNORE

T3 → IGNORE

9. Final Database State

After recovery:

A = 900 → T1 committed → REDO

B = Old Value → T2 uncommitted → Ignored

C = Old Value → T3 uncommitted → Ignored

Thus:

RECOVERED DATABASE

↓

A = 900

B = Old Value

C = Old Value

10. Deferred Update vs Immediate Update

Feature	Deferred Update	Immediate Update
Updates written before commit	No	Yes
REDO required	Yes	Yes
UNDO required	No	Yes
Uncommitted transaction	Ignored	Rolled back
Recovery method	NO-UNDO/REDO	UNDO/REDO

9)

Strict 2PL for Banking System

1. Introduction

In a banking system, multiple transactions may access the same account balance at the same time. For example:

- **T1** transfers money from one account to another.
- **T2** reads an account balance simultaneously.

Without proper concurrency control, T2 may read an incorrect or partially updated balance.

Therefore, we use **Strict Two-Phase Locking (Strict 2PL)**.

Main Rule of Strict 2PL

Acquire Lock

↓

Perform Read / Write

↓

COMMIT or ABORT



Release Lock

All exclusive locks are held until the transaction commits or aborts.

2. Banking Example

Assume two accounts:

Account A = ₹10,000

Account B = ₹5,000

Transaction **T1** transfers ₹2,000 from Account A to Account B.

Therefore:

T1:

$A = A - 2000$

$B = B + 2000$

After the transfer:

Account A = ₹8,000

Account B = ₹7,000

At the same time, **T2** wants to read the balance of Account A.

T2:

Read(A)

3. Lock Types Used

Shared Lock – S-Lock

Used for reading:

S(A)



Read Account A

Multiple transactions may read the same data item using compatible shared locks.

Exclusive Lock – X-Lock

Used for writing:

X(A)

↓

Read and Write Account A

When an exclusive lock exists, another transaction cannot read or write that item.

Existing Lock	Requested S-Lock	Requested X-Lock
S-Lock	Allowed	Not Allowed
X-Lock	Not Allowed	Not Allowed

4. Transaction T1 – Fund Transfer

T1 performs the transfer using exclusive locks.

T1:

LOCK-X(A)

LOCK-X(B)

READ(A)

A = A - 2000

WRITE(A)

READ(B)

B = B + 2000

WRITE(B)

COMMIT

UNLOCK(A)

UNLOCK(B)

The important Strict 2PL rule is:

X(A) and X(B)

↓

Held until COMMIT

↓

Locks Released

T1 cannot release its exclusive locks before committing.

5. Transaction T2 – Reading Balance

T2 wants to read Account A.

T2:

LOCK-S(A)

READ(A)

COMMIT

UNLOCK(A)

However, if T1 already holds an exclusive lock on A:

T1 → Holds X(A)

T2 → Requests S(A)

↓

Cannot Grant

↓

WAIT

6. How Strict 2PL Prevents Incorrect Reading

Suppose T1 has deducted ₹2,000 from A but has not yet added it to B.

During this intermediate state:

A = ₹8,000

B = ₹5,000

If T2 were allowed to read immediately, it could observe a transaction that is only partially completed.

Strict 2PL prevents this:

T1 holds X(A)

↓

T2 requests S(A)

↓

T2 WAITS

↓

T1 completes transfer

↓

T1 COMMIT

↓

T1 releases X(A)

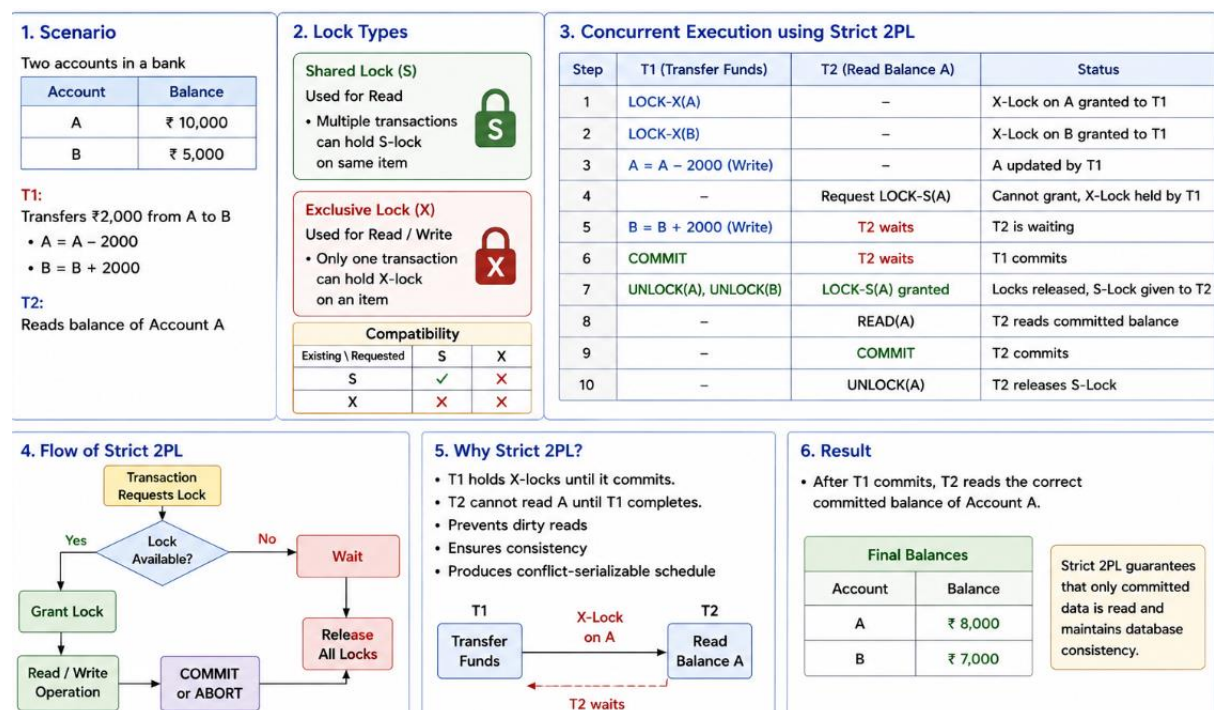
↓
T2 reads committed balance

Therefore, T2 reads:

$A = ₹8,000$

only after the transaction is completed and committed.

9. Concurrency Control Mechanism



10)

Query Optimization Using Heuristics

The question asks to write pseudocode using the heuristics **push down selection, project early, and order joins by smaller intermediate results.**

1. Introduction

Query optimization is the process of finding a more efficient way to execute a database query. Heuristic optimization uses rules to reduce the amount of data processed during query execution.

2. Pseudocode

Algorithm Heuristic_Query_Optimization(Query Q)

BEGIN

Step 1: Parse the query Q
Create the initial query tree

Step 2: Push Down Selection
Move each selection (σ) close to
the corresponding base relation
This reduces the number of rows early

Step 3: Project Early
Apply projection (π) early
Remove unnecessary attributes
This reduces the number of columns

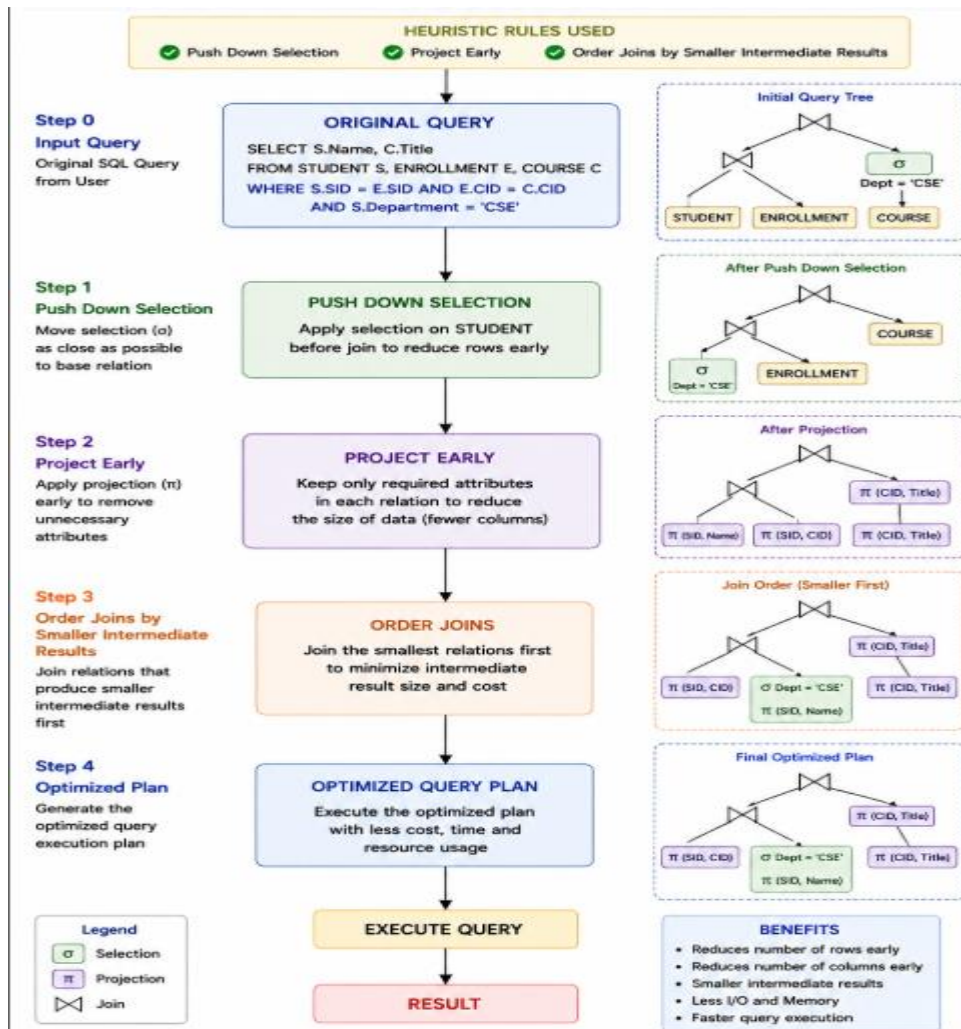
Step 4: Order Joins
Estimate intermediate result sizes
Perform joins producing smaller results first
This reduces processing cost

Step 5: Generate Optimized Query Plan

RETURN Optimized Query Plan

END

3. Optimization Flow



4. Example

Original Query:

$(\text{STUDENT} \bowtie \text{ENROLLMENT}) \bowtie \text{COURSE}$



Apply Selection:

$\sigma(\text{Department} = \text{'CSE'})(\text{STUDENT})$



Project Required Attributes:

$\pi(\text{SID}, \text{Name})$



Join Smaller Results First



Optimized Query Plan

5. Main Heuristic Rules

Heuristic	Purpose
Push Down Selection	Reduces rows early
Project Early	Reduces unnecessary columns
Order Smaller Joins First	Reduces intermediate results

Conclusion

These heuristics improve query performance by reducing unnecessary data processing.

Selection Early → Fewer Rows

Projection Early → Fewer Columns

Smaller Joins First → Less Processing Cost



Efficient Query Execution